

CINEMAX – DOCUMENTO DI COORDINAMENTO INTERNO

SINTESI OPERATIVA E RIPARTO RUOLI DEL TEAM DI SVILUPPO

Emesso da: Morad (Project Manager)

Destinatari: Giovanni, Harman, Ayman

Scadenza Rilascio: 1 Settimana (Dettaglio Esame)

Stato Project Board: Assegnata / Attiva

Il presente documento di coordinamento riassume in modo formale le responsabilità atomiche, i compiti implementativi e i vincoli strutturali da applicare immediatamente all'interno del progetto d'esame CineMax. Ciascun membro del team è tenuto a fare rigoroso riferimento ai paragrafi di propria pertinenza per l'avanzamento dei task sulla board di tracciamento comune.

1. Azioni e Responsabilità per Singolo Utente

Giovanni – Developer e Design Manager

L'attività iniziale e prioritaria prevede lo sviluppo della modellazione dei dati all'interno del pacchetto `core.models`. È richiesto il rilascio immediato delle classi ad oggetti pure `Utente.java`, `Proiezione.java` e `Prenotazione.java`, strutturate in modo rigoroso mediante l'incapsulamento di campi privati e relativi metodi getter e setter pubblici. Questo blocco costituisce un prerequisito critico e vincolante per l'avanzamento dei moduli successivi.

In merito alla logica di business (`core.services`), Giovanni si occuperà dell'implementazione degli algoritmi di filtraggio logico per la ricerca avanzata e combinata delle proiezioni, estendendo i criteri per titolo, genere, regista, prezzo e fascia oraria. Avrà inoltre la responsabilità di strutturare il controllo del palinsesto sul metodo di aggiunta, per impedire che una nuova proiezione generi sovrapposizioni temporali e orarie all'interno del medesimo file di sala. Per quanto concerne lo strato di presentazione (`tui`), svilupperà l'intero albero logico dei menu operativi a terminale, implementando la corretta separazione delle funzionalità disponibili secondo il livello di accesso dell'utente (Guest, Cliente, Proiezionista, Bigliettaio).

Harman – Developer e System Architect

L'ambito operativo principale di Harman riguarda lo sviluppo del layer di persistenza locale all'interno del pacchetto `data.csv`. Si occuperà di implementare la logica di tokenizzazione e parsing delle stringhe per l'analisi e la corretta scrittura e sincronizzazione sui file strutturati `film.csv`, `proiezioni.csv` e `utenti.csv`.

Sul fronte della sicurezza delle credenziali, Harman svilupperà l'algoritmo di cifratura delle password utente mediante la funzione di hashing SHA-256 combinata con una tecnica di salting, garantendo che nessuna credenziale venga memorizzata in chiaro all'interno della persistenza. Completerà la propria area d'azione mediante la stesura della classe `IDGenerator`, deputata a scansionare i file CSV esistenti all'avvio dell'applicazione per identificare il valore massimo degli indici e generare in maniera puramente sequenziale, atomica e univoca gli identificativi dei nuovi record inseriti.

Ayman – Quality Assurance e Tester

La responsabilità primaria di Ayman risiede nella verifica sistematica della stabilità del backend. Si occuperà della stesura di un'ampia suite di casi di test automatizzati mediante il framework JUnit, focalizzandosi in particolar modo sulla validazione delle regole di business e sulla gestione dei casi limite (edge cases), quali tentativi di login errati, formati di data non conformi e conflitti orari nel palinsesto.

Avrà il compito di condurre un'analisi di robustezza della Terminal User Interface, presidiando tutti i canali di input dello Scanner per intercettare preventivamente potenziali anomalie bloccanti come la `InputMismatchException`. Infine, coordinerà la redazione della cartella `doc`, occupandosi del confezionamento e della verifica dei manuali utente e tecnici finali necessari per la consegna ufficiale del progetto.

2. Specifica dei Requisiti Tecnici Transazionali

Al fine di garantire l'integrità e la consistenza delle informazioni scritte su disco, il sistema deve rispettare in modo nativo determinati vincoli operativi. La tabella sottostante mappa in modo analitico i moduli funzionali e i relativi controlli bloccanti applicati all'applicazione:

Modulo Operativo	Attore Abilitato	Vincolo Tecnico e Regola di Business
Programmazione Palinsesto	Proiezionista	Controllo preventivo delle sovrapposizioni orarie in sala (calcolo basato su ora inizio + durata del film).
Manutenzione Sessioni	Proiezionista	Modifica o eliminazione di una proiezione inibita in presenza di almeno una prenotazione attiva ad essa collegata.
Emissione Biglietti	Cliente / Bigliettaio	Verifica in tempo reale della capienza massima della sala monosala, bloccata tassativamente a 200 posti totali.
Storno Prenotazioni	Cliente	Cancellazione o modifica permessa esclusivamente se la data dell'evento è strettamente successiva al tempo corrente del sistema.

Ogni operazione di inserimento prenotazione deve calcolare dinamicamente il numero di posti residui nella sala monosala applicando la seguente espressione matematica:

$$\text{Posti Disponibili} = 200 - \Sigma \text{Biglietti Già Prenotati per la Proiezione}$$

Nel caso in cui l'input quantitativo inserito dall'utente superi la quota dei posti liberi calcolati, il backend deve rigettare la richiesta sollevando una `IllegalArgumentException`, impedendo la corruzione dei file di testo e notificando il blocco all'interfaccia.

Politica di Gestione delle Eccezioni: Tutti gli errori di I/O derivanti da file CSV corrotti o non raggiungibili devono essere intercettati internamente al layer dei dati. L'errore deve essere convertito in una stringa di avviso chiara per l'utente ("Errore di lettura file. Operazione annullata"), consentendo alla TUI di rimanere attiva senza interrompere bruscamente l'esecuzione del software.

3. Workflow di Collaborazione e Regole per il Team

Per garantire la massima stabilità dei sorgenti, sono istituite tre direttive tassative per la cooperazione sul repository remoto Git:

- **Branching Rigido:** È vietato inviare modifiche o porzioni di codice direttamente sui rami principali `main` o `develop`. Ogni sviluppo deve risiedere su un branch locale dedicato (es. `feature/tui-menu`).
- **Integrazione via Pull Request:** Al completamento di ogni singolo task, lo sviluppatore deve aprire una Pull Request verso il ramo `develop`. Il Project Manager verificherà la corretta compilazione in ambiente locale prima di autorizzare il merge definitivo.
- **Allineamento Quotidiano:** È fatto obbligo a ciascun membro del team di eseguire un comando di sincronizzazione `git pull` dal branch `develop` prima di avviare ogni sessione di codice, minimizzando l'insorgenza di conflitti sui file condivisi.